

BriskSeed: Continuous Query Acceleration for Dynamic ANNS

Your Name

Your Institution

May 9, 2026

Talk Roadmap (7-Step Logic)

- 1 What is the problem?
- 2 Why it matters?
- 3 Why existing work fails?
- 4 What is our key idea?
- 5 What is the key design?
- 6 What is the experimental result?
- 7 What is the takeaway?

1) What is the Problem?

Problem setting: Dynamic ANNS under continuous updates

- Dataset evolves round by round: update batch $\mathcal{U}_r$ then query batch $\mathcal{Q}_r$.
- Goal is not only low query latency, but high end-to-end throughput with maintenance cost included.
- Need to maximize

$$\text{QPS}(R) = \frac{\sum_{r=1}^R |\mathcal{Q}_r|}{T_s(R) + T_m(R)}$$

under a recall constraint relative to baseline.

Core challenge: keep acceleration *continuously valid* as the index topology keeps changing.

2) Why it Matters?

- ANNS is a core primitive in RAG, recommendation, anomaly detection, and fraud detection.
- In streaming scenarios, update rates are high and decision latency budgets are strict.
- Throughput decay over time directly reduces system utility and can cause real business risk.

Empirical motivation from your paper:

- Cross-round top- k overlap is often high.
- Query-hit frequencies are long-tail skewed.
- Per-query stability is heterogeneous.

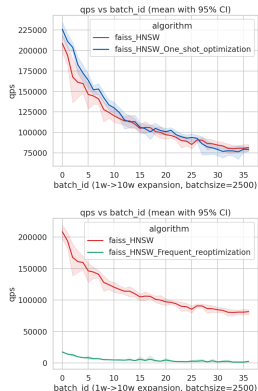
3) Why Existing Work Fails?

Static acceleration assumptions break in dynamic workloads

- Layout/quantization/hints are optimized for snapshots, not evolving indices.
- One-shot optimization gains decay as updates accumulate.
- Frequent global re-optimization is too expensive and hurts end-to-end QPS.

Dilemma:

- No maintenance $\Rightarrow$ stale guidance.
- Aggressive maintenance $\Rightarrow$ excessive overhead.



4) What Is Our Key Idea?

Turn one-shot hinting into an online reuse–validate–refresh loop.

- Reuse past successful search outcomes as compact **seeds**.
- Validate reused candidates before acceptance.
- Refresh/evict seeds online so memory tracks workload and drift.
- Keep the mechanism backend-agnostic via plugin adapters.

Intuition:

- Historical outcomes are not final answers.
- They are warm-start guidance with bounded risk.

5) What Is the Key Design? (Architecture)

Three co-designed layers

1 Seed Storage Layer

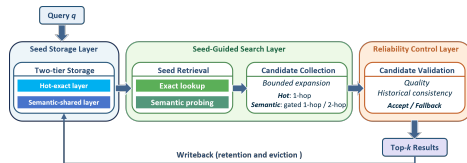
- Two-tier memory: hot-exact + semantic-shared.
- Seed record: $r = \langle \mathbf{z}, n_b, \beta, \rho \rangle$.

2 Seed-Guided Search Layer

- Retrieve seed by exact key or bounded semantic probing.
- Recover candidates with one-hop / guarded two-hop expansion.

3 Reliability Control Layer

- Accept only if quality and consistency both pass.
- Otherwise immediate fallback to baseline ANN search.



5) What Is the Key Design? (Pipeline and Guarantees)

Bounded online pipeline per query

- Retrieval cost: $O(Pm)$.
- Recovery cost: $O(kM)$ or $O(kM^2)$.
- Validation + writeback: bounded w.r.t. dataset size N .

Expected latency

$$T_{\text{BriskSeed}} = T_{\text{reuse}} + (1 - p_{\text{acc}})T_b, \quad T_{\text{reuse}} = O(Pm + kM^2)$$

Benefit condition

$$T_{\text{reuse}} < p_{\text{acc}} T_b$$

Consequence: graceful degradation under drift, and speedup when reuse remains sufficiently valid.

6) What Is the Experimental Result? (Placeholder)

To be filled after experiments are finalized

- Main metrics:
 - End-to-end QPS across rounds
 - Recall@ k vs baseline
 - Maintenance overhead ratio
- Main baselines:
 - Backend native mode
 - Existing dynamic ANNS backends (same benchmark protocol)
- Ablation plan:
 - Two-tier storage vs single-tier
 - Validation on/off
 - One-hop vs guarded two-hop

[Insert plots/tables here]

7) What Is the Takeaway?

- Dynamic ANNS acceleration is an **efficiency-maintenance** problem, not only a search-speed problem.
- BriskSeed turns static hinting into **online reusable seed management**.
- The design is **bounded**, **reliable**, and **backend-agnostic**.
- Practical value: better end-to-end throughput while preserving competitive recall under continuous updates.

Thank you!
Questions?